

Pract9_Clustering

October 8, 2026

```
[4]: #1.load the dataset
import pandas as pd
import matplotlib.pyplot as plt

df=pd.read_excel("Mall Customers.xlsx")

print(df.head())
print("Shape:",df.shape)
print("Missing values:")
print(df.isnull().sum)
```

	CustomerID	Gender	Age	Education	Marital Status	Annual Income (k\$)	\
0	1	M	19	High School	Married	15	
1	2	M	21	Graduate	Single	15	
2	3	F	20	Graduate	Married	16	
3	4	F	23	High School	Unknown	16	
4	5	F	31	Uneducated	Married	17	

	Spending Score (1-100)
0	39
1	81
2	6
3	77
4	40

Shape: (200, 7)

Missing values:

<bound method DataFrame.sum of
Marital Status \

	CustomerID	Gender	Age	Education	Marital Status	\
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
..	...	...	...	...	...	...
195	False	False	False	False	False	False
196	False	False	False	False	False	False
197	False	False	False	False	False	False
198	False	False	False	False	False	False

```
199      False  False  False      False      False
```

```

      Annual Income (k$)  Spending Score (1-100)
0                False                False
1                False                False
2                False                False
3                False                False
4                False                False
..                ...
195              False                False
196              False                False
197              False                False
198              False                False
199              False                False

```

```
[200 rows x 7 columns]>
```

```
[5]: #2.Select the features
X = df[["Annual Income (k$)","Spending Score(1-100)"]]
print(X.head())
```

```

-----
KeyError                                Traceback (most recent call last)
Cell In[5], line 2
      1 #2.Select the features
----> 2 X = df[["Annual Income (k$)","Spending Score(1-100)"]]
      3 print(X.head())

File /usr/lib/python3/dist-packages/pandas/core/frame.py:3899, in DataFrame.
    ↪ __getitem__(self, key)
      3897     if is_iterator(key):
      3898         key = list(key)
-> 3899     indexer = self.columns._get_indexer_strict(key, "columns")[1]
      3901 # take() does not accept boolean indexers
      3902 if getattr(indexer, "dtype", None) == bool:

File /usr/lib/python3/dist-packages/pandas/core/indexes/base.py:6115, in Index.
    ↪ _get_indexer_strict(self, key, axis_name)
      6112 else:
      6113     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
-> 6115 self._raise_if_missing(keyarr, indexer, axis_name)
      6117 keyarr = self.take(indexer)
      6118 if isinstance(key, Index):
      6119     # GH 42790 - Preserve name from an Index

File /usr/lib/python3/dist-packages/pandas/core/indexes/base.py:6179, in Index.
    ↪ _raise_if_missing(self, key, indexer, axis_name)

```

```

6176     raise KeyError(f"None of [{key}] are in the [{axis_name}]")
6178 not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique())
-> 6179 raise KeyError(f"{not_found} not in index")

```

```

KeyError: "['Spending Score(1-100)'] not in index"

```

```

[6]: X = df[["Annual Income (k$)", "Spending Score (1-100)"]]

print(X.head())

```

	Annual Income (k\$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

```

[7]: #3.Scale the feature
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(X_scaled[:5])

```

```

[[-1.73899919 -0.43480148]
 [-1.73899919  1.19570407]
 [-1.70082976 -1.71591298]
 [-1.70082976  1.04041783]
 [-1.66266033 -0.39597992]]

```

```

[8]: #4.Elbow Method
from sklearn.cluster import KMeans

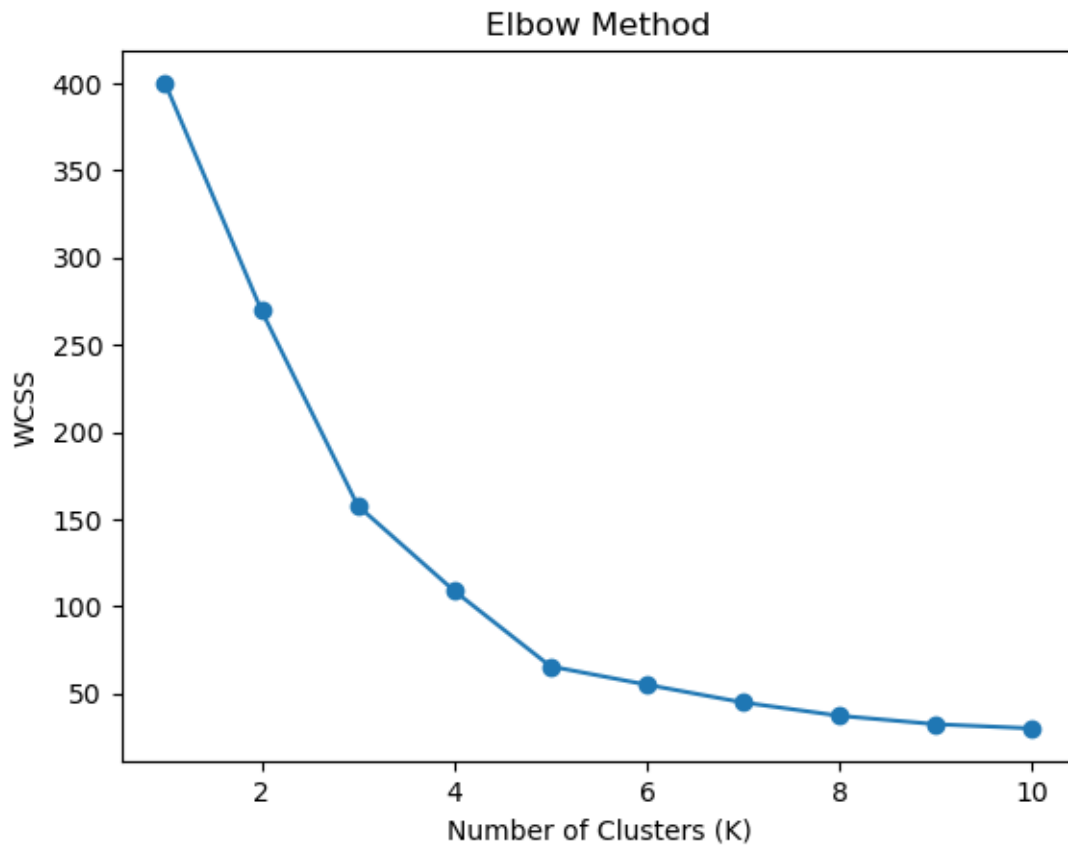
wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

print(wcss)
plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()

```

```
[400.00000000000006, 269.69101219276394, 157.70400815035947, 108.92131661364355,
65.56840815571681, 55.057348270386, 44.86475569922557, 37.22818767758587,
32.39226763033117, 29.981897788243693]
```



```
[9]: # Step 5: Apply K-Means Clustering
```

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
```

```
kmeans.fit(X_scaled)
```

```
[9]: KMeans(n_clusters=5, n_init=10, random_state=42)
```

```
[10]: # Step 5: Apply K-Means Clustering
```

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
```

```
kmeans.fit(X_scaled)
```

```
print("Cluster Labels:")
```

```
print(kmeans.labels_)
```

```
print("Cluster Centers:")
print(kmeans.cluster_centers_)
```

Cluster Labels:

```
[4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4
 2 4 2 4 2 4 0 4 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 1 3 1 0 1 3 1 3 1 0 1 3 1 3 1 3 1 3
 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3
 1 3 1 3 1 3 1 3 1 3 1 3 1 3 1]
```

Cluster Centers:

```
[[-0.20091257 -0.02645617]
 [ 0.99158305  1.23950275]
 [-1.32954532  1.13217788]
 [ 1.05500302 -1.28443907]
 [-1.30751869 -1.13696536]]
```

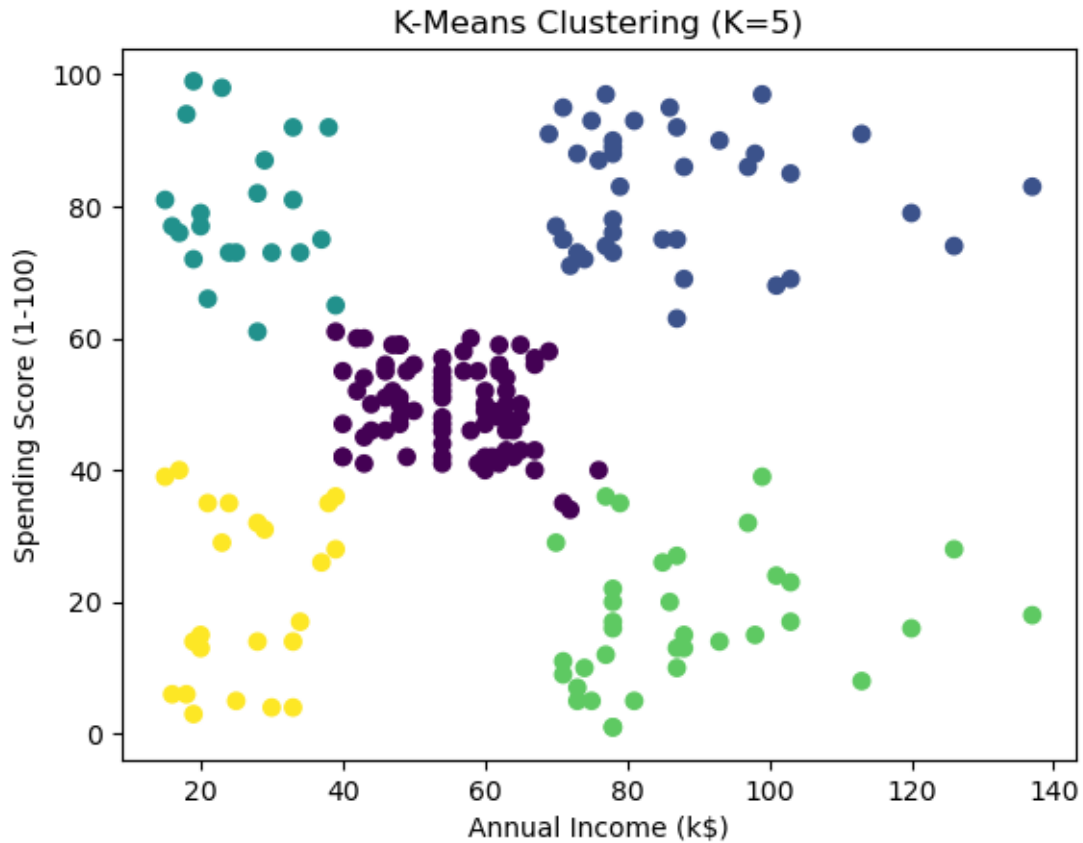
[11]: *# Step 5: Apply K-Means Clustering*

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)

kmeans.fit(X_scaled)

# Visualize the clusters
plt.scatter(
    X["Annual Income (k$)"],
    X["Spending Score (1-100)"],
    c=kmeans.labels_
)

plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("K-Means Clustering (K=5)")
plt.show()
```

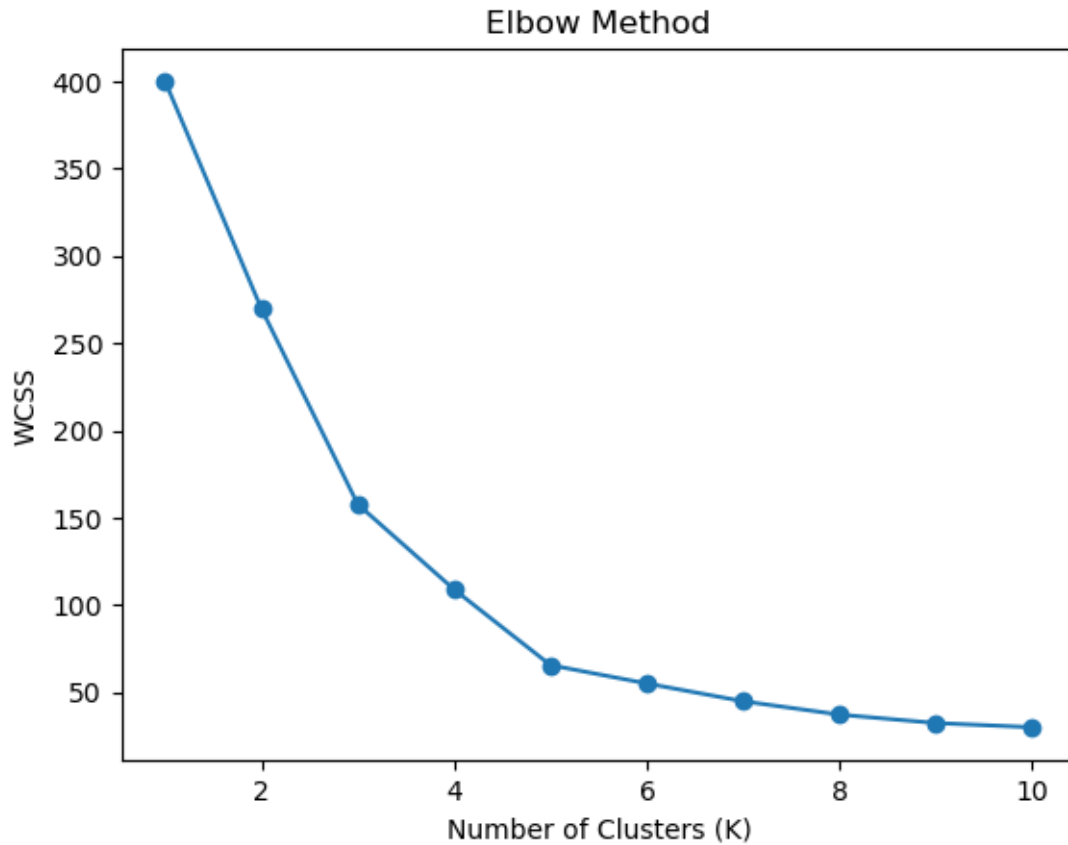


[12]: *# Step 4: Find the optimal number of clusters using Elbow Method*

```
wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

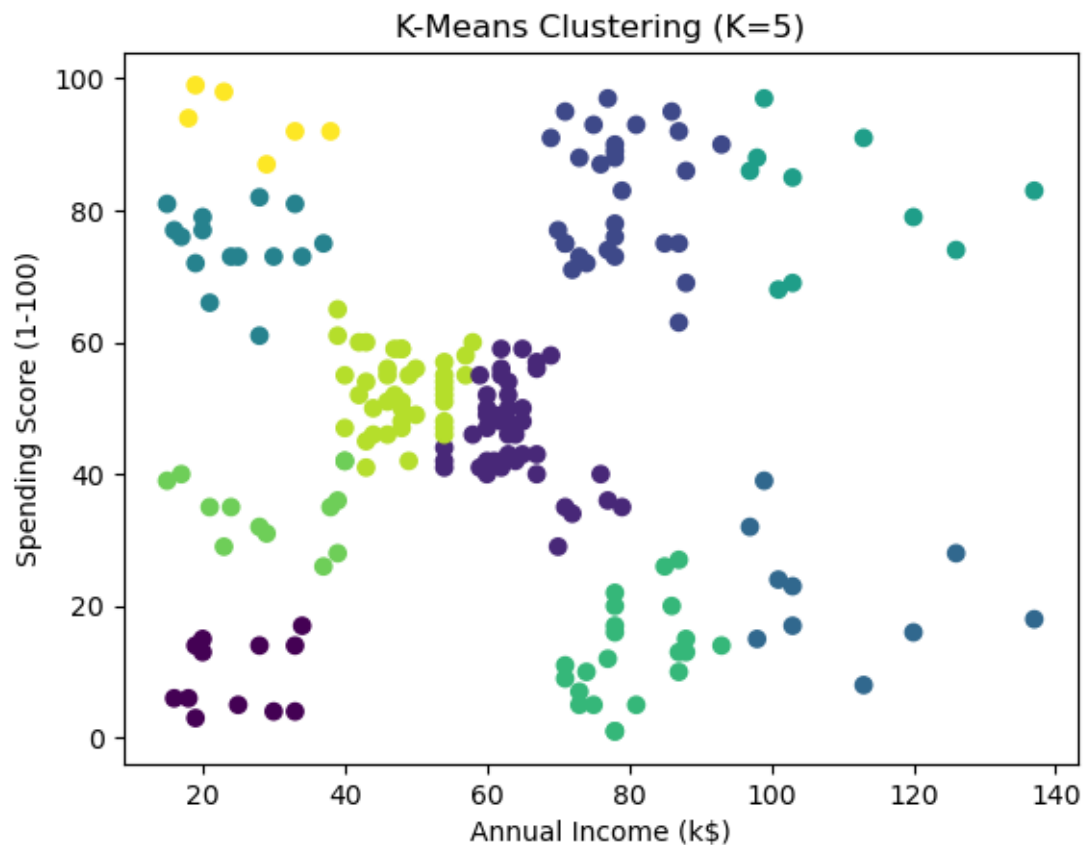
plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```



```
[13]: # Step 6: Visualize K-Means Clusters

plt.scatter(
    X["Annual Income (k$)"],
    X["Spending Score (1-100)"],
    c=kmeans.labels_
)

plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("K-Means Clustering (K=5)")
plt.show()
```



```
[14]: # Step 7: Assign Cluster Labels
```

```
df["Cluster"] = kmeans.labels_  
  
print(df.head())
```

	CustomerID	Gender	Age	Education	Marital Status	Annual Income (k\$)	\
0	1	M	19	High School	Married	15	
1	2	M	21	Graduate	Single	15	
2	3	F	20	Graduate	Married	16	
3	4	F	23	High School	Unknown	16	
4	5	F	31	Uneducated	Married	17	

	Spending Score (1-100)	Cluster
0	39	7
1	81	4
2	6	0
3	77	4
4	40	7

```
[15]: # Step 8: Convert Cluster Centers to Original Scale

centers = scaler.inverse_transform(kmeans.cluster_centers_)

print("Cluster Centers:")
print(centers)
```

```
Cluster Centers:
[[ 24.58333333   9.58333333]
 [ 63.72093023  46.1627907 ]
 [ 78.55172414  82.17241379]
 [109.7         22.         ]
 [ 24.46666667  74.6         ]
 [109.7         82.         ]
 [ 80.18181818  12.68181818]
 [ 30.          34.61538462]
 [ 48.1         52.675        ]
 [ 26.66666667  93.66666667]]
```

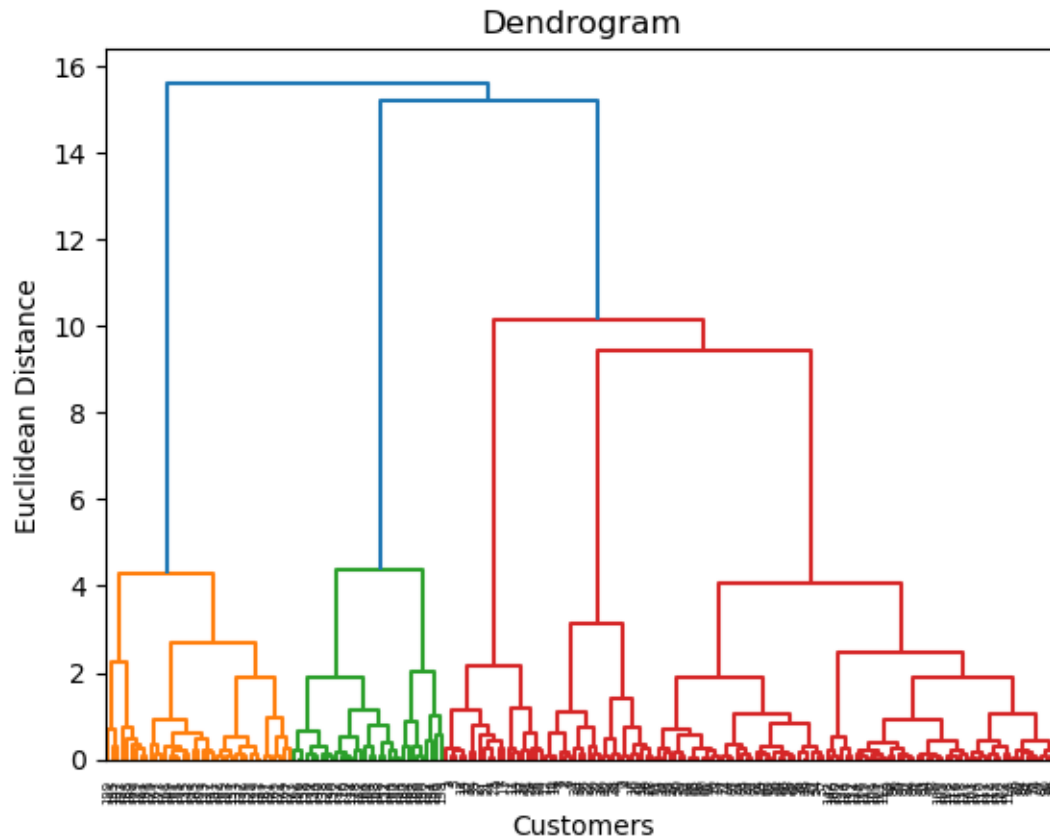
```
[16]: # Step 1: Import libraries for Dendrogram

import scipy.cluster.hierarchy as sch
import matplotlib.pyplot as plt
```

```
[17]: # Step 2: Create Dendrogram

dendrogram = sch.dendrogram(
    sch.linkage(X_scaled, method="ward")
)

plt.title("Dendrogram")
plt.xlabel("Customers")
plt.ylabel("Euclidean Distance")
plt.show()
```



```
[18]: # Step 3: Select Number of Clusters
```

```
k = 5
```

```
[19]: # Step 4: Apply Hierarchical Clustering
```

```
from sklearn.cluster import AgglomerativeClustering
```

```
hc = AgglomerativeClustering(n_clusters=5, linkage="ward")
```

```
y_hc = hc.fit_predict(X_scaled)
```

```
print(y_hc)
```

```
[4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4 3 4
 3 4 3 4 3 4 2 4 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 1 2 1 2 1 0 1 0 1 2 1 0 1 0 1 0 1 2 1 0 1 2 1
 0 1 0 1 0 1 0 1 0 1 0 1 2 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0
 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1]
```

```
[20]: # Step 6: Add Cluster Labels
```

```
df["Hierarchical_Cluster"] = y_hc

print(df.head())
```

	CustomerID	Gender	Age	Education	Marital Status	Annual Income (k\$)	\
0	1	M	19	High School	Married	15	
1	2	M	21	Graduate	Single	15	
2	3	F	20	Graduate	Married	16	
3	4	F	23	High School	Unknown	16	
4	5	F	31	Uneducated	Married	17	

	Spending Score (1-100)	Cluster	Hierarchical_Cluster
0	39	7	4
1	81	4	3
2	6	0	4
3	77	4	3
4	40	7	4

```
[21]: # Step 7: Display Cluster Centers
```

```
df["Hierarchical_Cluster"] = y_hc

centers = df.groupby("Hierarchical_Cluster")[
    ["Annual Income (k$)", "Spending Score (1-100)"]
].mean()

print("Cluster Centers:")
print(centers)
```

Cluster Centers:

	Annual Income (k\$)	Spending Score (1-100)
Hierarchical_Cluster		
0	89.406250	15.593750
1	86.538462	82.128205
2	55.811765	49.129412
3	25.095238	80.047619
4	26.304348	20.913043

```
[22]: # Step 8: Compare Clustering Results
```

```
print("K-Means Clusters:")
print(df["Cluster"].value_counts())

print("\nHierarchical Clusters:")
print(df["Hierarchical_Cluster"].value_counts())
```

K-Means Clusters:

```
Cluster
1      43
8      40
2      29
6      22
4      15
7      13
0      12
3      10
5      10
9       6
Name: count, dtype: int64
```

```
Hierarchical Clusters:
Hierarchical_Cluster
2      85
1      39
0      32
4      23
3      21
Name: count, dtype: int64
```

```
[ ]:
```